

Original articles

Improved constrained physics-informed neural networks (ICPINNs) to solve PDE and its application to option pricing

Jianguo Tan^{*,} Xingyu Zhang

School of Mathematical Sciences, Tiangong University, Tianjin, 300387, PR China



ARTICLE INFO

Keywords:

Deep learning
Hard-constraint and soft-constraint boundary conditions
Neural network
Option pricing

ABSTRACT

Previous deep learning methods, whether using hard or soft constraints, have often exhibited slow convergence when dealing with more than two types of boundary conditions (BCs). In general, option pricing partial differential equations (PDEs) encompass a mix of Dirichlet and non-Dirichlet BCs [1]. Hence, we propose a new method, the ICPINN method, which combines the soft-constraint and hard-constraint BCs to impose constraints based on different types of BCs for PDE. The ICPINN method is highly accurate for solving PDEs with Dirichlet and non-Dirichlet BCs. To verify the capacity of the ICPINN method, we apply it to the Black-Scholes (BS) model, the Heston option pricing PDE, and the three-asset BS model. The results show that the numerical solutions obtained by the ICPINN method can accurately converge to the exact solutions of the three financial option pricing PDEs. At the same time, the ICPINN method is more accurate. It converges faster than Physics-Informed Neural Networks (PINNs), PINNs with hard constraints (hPINNs), and boundary-safe PINNs.

1. Introduction

Traditional option pricing methods involving PDEs can be traced back to the seminal BS model [2,3]. As theoretical advancements approach practical implementation, numerous option pricing PDEs have emerged, including the Heston model [4–6], multi-asset BS models [7,8], and jump diffusion models by Merton [9] and Kou [10]. Early studies predominantly employed Dirichlet BCs; however, subsequent research suggests that incorporating derivative information through non-Dirichlet conditions at infinity could enhance numerical accuracy. We must emphasize that although the non-Dirichlet boundary conditions in the financial examples discussed in this paper do not affect the analytical solution of the PDE, as noted in references [11] and [12], they can enhance the accuracy and convergence of numerical solutions. Therefore, this paper will investigate the PINN algorithm involving non-Dirichlet boundary conditions from the perspective of numerical solutions. Considering both this perspective and the essential initial condition given by the payoff function, option pricing PDEs should generally be characterized as mixed boundary value problems.

Conventional numerical methods for solving these PDEs – particularly finite difference methods [13,14], Fourier transform approaches [15], Monte Carlo (MC) simulations [16], spectral methods [17], and operator splitting techniques [18] – inevitably encounter challenges such as mesh generation, convergence uncertainty, and computational intensity [19]. These limitations underscore the need for developing more efficient and robust solution methodologies for PDEs in option pricing contexts.

The evolution of computational hardware and deep learning frameworks like TensorFlow and PyTorch has enabled novel approaches to differential equation solving. Recent developments encompass neural network solutions for ordinary differential equations (ODEs) [20,21], PDEs [22–24], stochastic differential equations (SDEs) [25], fractional differential equations (FDEs) [26,27], and PIDEs [19,28].

* Corresponding author.

E-mail address: tanjg@tju.edu.cn (J. Tan).

Deep learning methods have demonstrated significant potential for option pricing in recent years. Villarino et al. proposed the boundary-safe PINN method, which addresses the challenge of BC imposition by reformulating the residuals associated with Dirichlet or Neumann conditions. They applied the algorithm to a high-dimensional BS model to demonstrate the effectiveness of the method [1]. Bai et al. [29] discovered that incorporating a slope recovery term for adaptive activation functions in the loss function accelerates convergence when using the PINNs method in solving the BS models. Galu et al. proposed the deep parametric PDE method, which employs a single neural network to approximate solutions for parametric PDE families, and successfully applied it to the multivariate BS model [30]. Hainaut et al. proposed an option pricing method based on PINNs, which is applied to European option valuation under the Heston model [31].

However, in most approaches, these constraints are treated as soft methods. Enforcing all BCs via preprocessing is referred to as using hard-constrained BCs [32]. The most widely used preprocessing method is the one proposed by Lagaris et al. for constructing trial solutions [33]. The PINN method incorporating these hard constraints is referred to as hPINN by Lu et al. [32]. Solving PDEs for option pricing using the PINN method is a widely used approach [29,34,35]. Nonetheless, many scholars have used the hPINN method for option pricing and regarded it as an accelerated convergence method of the PINN method [36–38]. However, most focus on BS models that use only Dirichlet BCs. When non-Dirichlet BCs are involved, certain challenges arise. Firstly, the trial solution must include the derivative of the neural network's output with respect to its input [33]. This requirement increases the amount of gradient backpropagation calculations during training. Secondly, when the BCs become more complex, designing trial solutions is very difficult. Therefore, the ICPINN method is proposed in this paper to effectively handle situations similar to the Heston option pricing PDE with both Dirichlet and non-Dirichlet BCs.

The most significant contribution of this article is the proposal of a novel improved constraint method. For PDEs with Dirichlet and non-Dirichlet BCs, ICPINN uses a combination of hard and soft constraints rather than a single method. In other words, we impose hard constraints on Dirichlet BCs, while for non-Dirichlet BCs, we adopt soft constraints during optimization. Meanwhile, the constraints used by ICPINN differ from those of other deep learning methods, resulting in differences in the basic form of the loss function. Therefore, when the convergence speed cannot be directly compared through the decline of the loss function, we adopt the Mean Squared Error (MSE) function for evaluation, and its expression is detailed in Section 3.

The paper is organized as follows. In Section 2, we introduce the algorithm of the PINN method with soft-constrained BCs and then present the PINN method with hard-constrained BCs. We then provide a detailed explanation of the ICPINN method, which integrates both soft and hard constraints to handle different types of BCs for PDEs. In Section 3, we define our new convergence evaluation criteria and demonstrate the effectiveness and convergence of the ICPINN method for the BS model, the Heston option pricing PDE, and the three-asset BS model. Finally, we conclude the paper in Section 4.

All code in this paper is implemented using Python 3.10.0 and TensorFlow-gpu 2.7.0. Numerical experiments were performed on a laptop equipped with an AMD Ryzen 7 7840H processor, a GeForce RTX 4060 graphics card, and 16.0 GB of RAM.

2. Methodology

This section is divided into three parts: the first two parts introduce the PINN and the hPINN methods, and the final part focuses on the detailed introduction of the ICPINN method. Based on PDEs with Dirichlet and non-Dirichlet BCs, we employ the ICPINN method to handle these constraints. Rather than using a single constraint method, the approach is tailored according to the type of BC.

We begin by formalizing the problem geometry: Let $\Omega \subset \mathbb{R}^d$ denote a bounded domain with spatial coordinates $\mathbf{x} = (x_1, \dots, x_d)^\top$. The scalar-valued function $u : \Omega \rightarrow \mathbb{R}$ under consideration maps the domain to real numbers through $\mathbf{x} \mapsto u(\mathbf{x})$. The function $u(\mathbf{x})$ is the solution of the PDE, defined as follows:

$$\mathcal{L}u = 0, \quad \forall \mathbf{x} \in \Omega, \quad (2.1)$$

subject to mixed BCs on partitioned boundaries:

$$B_1 u := u(\mathbf{x}) - g_1(\mathbf{x}) = 0, \quad \mathbf{x} \in \partial\Omega_1, \quad (2.2a)$$

$$B_2 u := \mathcal{B}u - g_2(\mathbf{x}) = 0, \quad \mathbf{x} \in \partial\Omega_2, \quad (2.2b)$$

where the boundary decomposition satisfies $\partial\Omega_1 \cup \partial\Omega_2 = \partial\Omega$ with $\partial\Omega_1 \cap \partial\Omega_2 = \emptyset$. Here $\mathcal{L}$ and $\mathcal{B}$ represent differential operations. The general boundary operator B is specifically instantiated as B_1 for Dirichlet BCs and B_2 for non-Dirichlet BCs.

2.1. Physics-informed neural networks

A feedforward neural network with a total of $L + 2$ layers consists of L hidden layers, an input layer (denoted as the 0-th layer), and an output layer (denoted as the $(L + 1)$ -th layer). The l -th layer consists of n_l neurons. The output of each layer is utilized as the input for the subsequent layer, and the corresponding affine transformation is described below:

$$L_l(\mathbf{x}^{(l-1)}) = \mathbf{w}^{(l)} \mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}, \quad l = 1, 2, \dots, L + 1, \quad (2.3)$$

$$\text{NN}(\mathbf{x}; \theta) = (L_{L+1} \circ \sigma \circ L_L \circ \dots \circ \sigma \circ L_1)(\mathbf{x}^0). \quad (2.4)$$

Here θ represents the trainable parameters of the neural network, comprising $\{\mathbf{w}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L+1}$. Specifically, $\mathbf{w}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ denotes the weight matrix of the l th layer, and $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ denotes the bias vector of the bias of the l th layer. $\mathbf{x}^0$ denotes the input of the neural network. The function $L_l(\mathbf{x}^{(l-1)})$ describes the linear transformation at the l th layer, where $\mathbf{x}^{(l-1)}$ is the output from the $(l-1)$ -th layer. The composition of transformations and activations in Eq. (2.4) defines the overall output $u_\theta(\mathbf{x})$ of the neural network. Here, σ denotes the activation function applied element-wise.

For the PDE with BCs described by Eqs. (2.1) and (2.2), the solution $u(\mathbf{x})$ is approximated by the neural network output $\text{NN}(\mathbf{x}; \theta)$ as introduced in [23]. The objective is to optimize the parameters θ using a suitable optimization technique to ensure that the neural network output $\text{NN}(\mathbf{x}; \theta)$ effectively approximates $u(\mathbf{x})$.

To achieve this goal, the boundary residual $\text{Loss}_b(\theta)$, and PDE residual $\text{Loss}_r(\theta)$ are formulated by employing the mean square error:

$$\text{Loss}_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} [\mathcal{L}\text{NN}(\mathbf{x}_r^i; \theta)]^2, \quad (2.5)$$

$$\text{Loss}_b(\theta) = \frac{1}{N_{b_2}} \sum_{i=1}^{N_b} [\text{BNN}(\mathbf{x}_b^i; \theta)]^2, \quad (2.6)$$

where $\{\mathbf{x}_b^i\}_{i=1}^{N_b} \subset \partial\Omega_1 \cup \partial\Omega_2$ and $\{\mathbf{x}_r^i\}_{i=1}^{N_r} \subset \Omega$. Here N_b and N_r are the numbers of points for each respective set. Then the abstract expression of loss function $L_{\text{PINN}}(\theta)$ can be defined as:

$$L_{\text{PINN}}(\theta) = \lambda_r \text{Loss}_r(\theta) + \lambda_b \text{Loss}_b(\theta), \quad (2.7)$$

where λ_b and λ_r fixed penalty coefficients of the soft constraints, typically adjusted empirically to balance the different components of the total loss.

In the PINN method, all constraints are soft constraints, meaning that we do not require the BCs to be exactly satisfied; instead, we handle these constraints by the terms $\lambda_b \text{Loss}_b$ in the loss function (2.7). However, the presence of multiple terms in the loss function may induce convergence instability, while existing studies [32,36–38] demonstrate that hard-constrained BCs yield accelerated network convergence relative to soft-constrained formulations. Next, the details of PINN with hard constraints are shown.

2.2. Physics-informed neural networks with hard constraints

The main point of the hard-constraint BCs is that BCs need to be enforced before the training process. The most common way to enforce BCs is to construct trial solution $u_\theta(\mathbf{x})$, which can be used for all types of BCs, including Dirichlet, Neumann, Robin BCs [32]. Let the trial solution be formulated as follows:

$$u_\theta(\mathbf{x}) = A(\mathbf{x}) + B(\mathbf{x}) \cdot \phi_\theta(\mathbf{x}), \quad (2.8)$$

where $A(\mathbf{x})$, $B(\mathbf{x})$ are auxiliary functions. Typically, under the requirement that $u_\theta(\mathbf{x})$ satisfies all BCs, $\phi_\theta(\mathbf{x})$ is defined as a function that contains the neural network $\text{NN}(\mathbf{x}; \theta)$ and its associated derivative information $\text{NN}_{x_i}(\mathbf{x}; \theta)$, $(1 \leq i \leq d)$. This requirement arises because the trial solution $u_\theta(\mathbf{x})$ must satisfy the non-Dirichlet BCs (2.2b), namely:

$$\mathcal{B}u_\theta(\mathbf{x}) = \mathcal{B}A(\mathbf{x}) + \mathcal{B}(B(\mathbf{x}) \cdot \phi_\theta(\mathbf{x})) = g_2(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_2. \quad (2.9)$$

If we assume that $\phi_\theta(\mathbf{x})$ does not contain any derivative information of the neural network, then the expression $\mathcal{B}(B(\mathbf{x}) \cdot \phi_\theta(\mathbf{x}))$ must inevitably involve the unknown terms $\text{NN}(\mathbf{x}; \theta)$ and $\text{NN}_{x_i}(\mathbf{x}; \theta)$. Now, we need to adjust the expressions of the auxiliary functions $A(\mathbf{x})$ and $B(\mathbf{x})$ to directly eliminate the unknown terms $\text{NN}(\mathbf{x}; \theta)$ and $\text{NN}_{x_i}(\mathbf{x}; \theta)$ while ensuring that the Eq. (2.9) is satisfied. This is evidently a challenging task. However, if $\phi_\theta(\mathbf{x})$ is designed to incorporate the derivative information $\text{NN}_{x_i}(\mathbf{x}; \theta)$, allowing the $\mathcal{B}(B(\mathbf{x}) \cdot \phi_\theta(\mathbf{x}))$ term to internally cancel out the derivative terms, then the construction of $A(\mathbf{x})$ and $B(\mathbf{x})$ will be significantly simplified. Moreover, in practical scenarios, as non-Dirichlet BCs grow more complex, constructing a suitable ϕ to eliminate $\text{NN}_{x_i}(\mathbf{x}; \theta)$ remains highly challenging. For more detailed operations on constructing $\phi_\theta(\mathbf{x})$, please refer to literature [33].

The trial solution $u_\theta(\mathbf{x})$ can be tailored to satisfy all BCs (2.2) by setting up the following conditions:

$$A(\mathbf{x}) = g_1(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_1, \quad (2.10a)$$

$$B(\mathbf{x}) = 0, \quad \mathbf{x} \in \partial\Omega_1, \quad (2.10b)$$

$$\mathcal{B}u_\theta(\mathbf{x}, t) = g_2(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_2. \quad (2.10c)$$

Then we formulate the loss function based on the PDE (2.1) and the trial solution $u_\theta(\mathbf{x})$ as follows:

$$L_{h\text{PINN}}(\theta) = \text{Loss}_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} [\mathcal{L}u_\theta(\mathbf{x}_r^i)]^2. \quad (2.11)$$

Here, $\{\mathbf{x}_r^i\}_{i=1}^{N_r} \subset \Omega$ where N_r represents the number of points.

The primary objective in training the parameters θ is to minimize the loss function $L_{h\text{PINN}}(\theta)$. By doing so, we leverage the capacity of the feedforward neural network to approximate functions and achieve solutions that closely match those of the PDE.

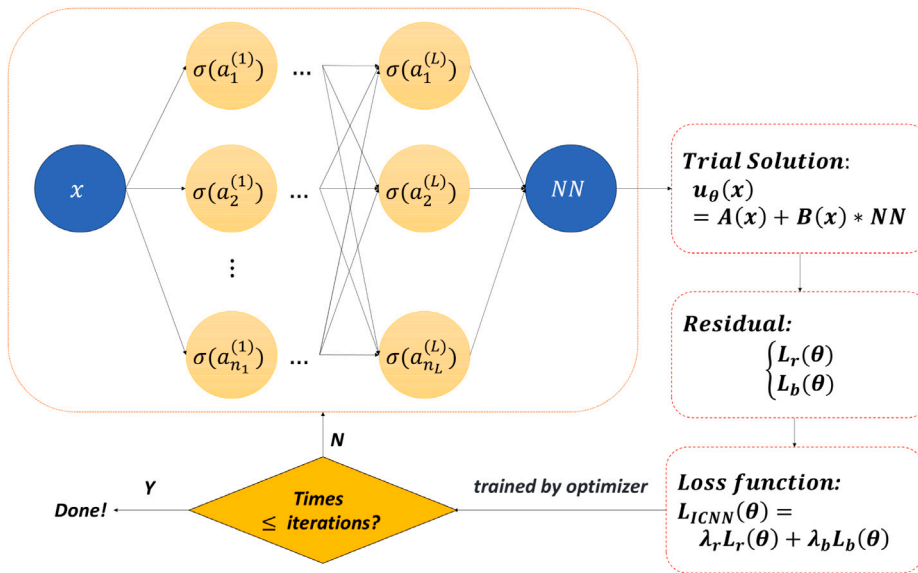


Fig. 2.1. Schematic of ICPINN.

The hPINN method is often regarded as an accelerated convergence scenario of the PINN method. However, as the BCs become more complex, the difficulty and formal complexity of constructing trial solutions increases. In situations where the BCs extend beyond Dirichlet conditions, the trial solution may incorporate derivative values of the neural network output with respect to the input [33]. Based on this situation, several challenges can arise:

- **Increasing computational load:** Each iteration necessitates additional computations to calculate derivatives from the output to the input, thereby directly augmenting time and memory consumption per iteration.
- **Arising gradient vanishing risk:** The trial solution's heavy reliance on input derivatives may lead to the disappearance of gradients, increasing the likelihood of convergence to local minima.
- **The issue of complexity in designing trial solutions:** When faced with multiple Neumann or even Robin BCs, the structure of the trial solution becomes notably intricate, adding to the complexity of the overall solution process.

To address these challenges, we propose an improved constrained neural network method.

2.3. Improved constrained physics-informed neural networks

For the ICPINN method, we adopt distinct strategies for BCs:

- For simple Dirichlet BCs, we enforce hard constraints by constructing trial functions that inherently satisfy the prescribed BCs from the onset of the training process.
- Moreover, for non-Dirichlet BCs, we opt for soft constraints to ensure smoother convergence.

Additionally, a schematic of the ICPINN method is depicted in Fig. 2.1, and the general process can refer to Table 1. The specific ICPINN algorithm comprises the following detailed procedure.

1. **Choosing the training set:** While uniformly distributed and equidistant sampling methods are common, we opt for Sobol sampling [39]. By applying Sobol sampling to the training interval, we obtain the requisite training sets, including the non-Dirichlet boundary dataset $\{x_b^i\}_{i=1}^{N_b}$ sampled from $\partial\Omega_2$ and the central area dataset $\{x_r^i\}_{i=1}^{N_r}$ sampled from Ω .
2. **Designing the trial solution $u_\theta(x)$:** The trial solution of the ICPINN method in this section differs from that of the previous Section 2.2. Regarding PDE (2.1) with the BCs (2.2), the basic form of the trial solution we construct is significantly simpler than the scenario with hard constraints on the BCs. The trial solution takes the form:

$$u_\theta(x) = A(x) + B(x) \cdot NN(x; \theta). \quad (2.12)$$

Here, $NN(x; \theta)$ represents the output of the neural network initialized using the hyperparameters in Table 3. $A(x)$ satisfies Eq. (2.10a), while $B(x)$ satisfies Eq. (2.10b). However, we do not require the trial solution $u_\theta(x)$ to satisfy Eq. (2.10c). Regarding the construction of auxiliary functions $A(x)$ and $B(x)$, we can implement them by the techniques in [33]. But in this article, we utilize a small neural network to fit $A(x)$ for approximating Dirichlet BCs, while setting $B(x)$ to 0 at the corresponding

Table 1

Pseudo-code of the ICPINN method.

Algorithm ICPINN	
Input:	Non-Dirichlet boundary data set $\{\mathbf{x}_b^i\}_{i=1}^{N_b}$, central area data set $\{\mathbf{x}_r^i\}_{i=1}^{N_r}$, the size of neural network, and set the number of iterations $\mathcal{N}$.
Output:	Solution of PDE.
1:	Initialize the weights and biases of the neural network and define them as θ .
2:	for $j = 1, 2, 3, \dots, \mathcal{N}$:
3:	Pass the training data set to the neural network and get the trial solution $u_{\theta_j}(\mathbf{x})$.
4:	Using the automatic differentiation (AD) technique we get $\mathcal{L}u_{\theta_j}(\mathbf{x}_r^i)$ and $\mathcal{B}u_{\theta_j}(\mathbf{x}_b^i)$.
5:	Build loss function $L_{\text{ICPINN}}(\theta_j) = \lambda_r \frac{1}{N_r} \sum_{i=1}^{N_r} [\mathcal{L}u_{\theta_j}(\mathbf{x}_r^i)]^2 + \lambda_b \frac{1}{N_b} \sum_{i=1}^{N_b} [\mathcal{B}u_{\theta_j}(\mathbf{x}_b^i) - g_2(\mathbf{x}_b^i)]^2$
6:	Obtain θ_{j+1} by minimizing the loss function.
7:	Update neural network using θ_{j+1} .

Table 2Pseudo-code for Approximating $A(\mathbf{x})$ using a small NN.

Algorithm for approximating $A(\mathbf{x})$	
Input:	Dirichlet boundary dataset $\{\mathbf{x}_{b0}^i\}_{i=1}^{N_{b0}}$, number of iterations $\mathcal{N}_{b0}$.
Output:	$A(\mathbf{x})$
1:	Initialize weights and biases of the neural network and define them as β .
2:	for $j = 1, 2, 3, \dots, \mathcal{N}_{b0}$:
3:	Pass training dataset to the neural network and get the approximation $A_{\beta_j}(\mathbf{x})$.
4:	Compute $B_1 A_{\beta_j}(\mathbf{x})$ via equation (2.2a).
5:	Construct loss function $L_A(\beta_j) = \frac{1}{N_{b0}} \sum_{i=1}^{N_{b0}} [B_1 A_{\beta_j}(\mathbf{x}_r^i)]^2$.
6:	Obtain β_{j+1} by minimizing the loss function.
7:	Update neural network parameters to β_{j+1} .

positions. The pseudocode for the process of fitting the function $A(\mathbf{x})$ using a small neural network is shown in Table 2. It is important to note that for the ICPINN algorithm, since we use a small neural network to approximate the Dirichlet BCs, the auxiliary function $A(\mathbf{x})$ does not strictly satisfy the Dirichlet BCs. Nonetheless, the underlying idea—enforcing the trial solution to satisfy the Dirichlet BCs before training—remains the same. Moreover, this approach avoids the need to design different auxiliary functions $A(\mathbf{x})$ for varying BCs. Further experimental results also demonstrate that even without strictly satisfying the Dirichlet BCs, our method still achieves better convergence compared to other similar deep learning approaches.

3. **Constructing the loss function** $L_{\text{ICPINN}}(\theta)$: For PDE (2.1) with BCs (2.2), the trial solution $u_{\theta}(\mathbf{x})$ has satisfied the condition (2.2a). We need to ensure that the trial solution $u_{\theta}(\mathbf{x})$ satisfies the PDE (2.1) and the non-Dirichlet condition (2.2b). The PDE residual $\text{Loss}_r(\theta)$, and the non-Dirichlet boundary residual $\text{Loss}_b(\theta)$ is defined as Eqs. (2.13) and (2.14). The overall loss function $L_{\text{ICPINN}}(\theta)$ is defined as Eq. (2.15), where λ_r and λ_b are the fixed penalty coefficients of the soft constraints

$$\text{Loss}_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} [\mathcal{L}u_{\theta}(\mathbf{x}_r^i)]^2, \quad (2.13)$$

$$\text{Loss}_b(\theta) = \frac{1}{N_b} \sum_{i=1}^{N_b} [\mathcal{B}u_{\theta}(\mathbf{x}_b^i) - g_2(\mathbf{x}_b^i)]^2, \quad (2.14)$$

$$L_{\text{ICPINN}}(\theta) = \lambda_b \text{Loss}_b(\theta) + \lambda_r \text{Loss}_r(\theta), \quad (2.15)$$

where $\{\mathbf{x}_r^i\}_{i=1}^{N_r} \subset \Omega$, $\{\mathbf{x}_b^i\}_{i=1}^{N_b} \subset \partial\Omega_2$, and N_r , N_b is the number of points.

4. **Minimizing the loss function and updating the parameters θ** : We optimize the parameters θ by minimizing the loss function (2.15), ensuring that the trial solution $u_{\theta}(\mathbf{x})$ effectively approximates the PDE solution $u(\mathbf{x})$. In this paper, we employ the Adam optimization method to minimize the loss function, as detailed in Table 3.

Finally, the main features of the ICPINN method can be summarized as follows:

- The primary application scenario of the ICPINN method is when the PDE has Dirichlet and non-Dirichlet BCs.

Table 3

The hyperparameters of the feedforward neural network.

Hyperparameters	Value
Number of hidden layers	8 layers
Number of neurons in each hidden layer	40
Activation function	Tanh
Regularization	l1_l2
Parameters initialization	glorot_normal
Learning rate	initial learning rate = 0.001
	decay rate = 0.75
	decay steps = 5000

Table 4

The hyperparameters of the feedforward neural network used for fitting auxiliary function.

Hyperparameters	Value
Number of hidden layers	3 layers
Number of neurons in each hidden layer	40
Activation function	Elu
Regularization	l1_l2
Parameters initialization	glorot_normal
Learning rate	learning rate = 0.005

- Unlike the PINN method, the ICPINN method does not consider the residual on the Dirichlet BCs. Additionally, compared to constructing the trial solution to satisfy all BCs, the trial solution of the ICPINN method has a simpler design and requires less computation.
- Specifically, when the BCs solely comprise Dirichlet conditions, our method aligns with the hPINN method. Conversely, when solely non-Dirichlet conditions are present, the ICPINN method resembles the PINN method.

3. Numerical experiments

In this section, we apply the ICPINN method to the BS model, the Heston option pricing PDE, and the three-asset BS model. We aim to demonstrate the advantages of the ICPINN method by comparing it with the different deep learning methods. Hence, we compare the results obtained by the ICPINN method with the PINN, the hPINN and the boundary-safe PINN method. However, for the ICPINN method, there is no need to consider the residual of the Dirichlet BCs, which also leads to a different loss function structure in the ICPINN method compared to other deep learning approaches. Therefore, we are not able to directly compare the convergence speed as described in other papers using the reduction process of the loss function. To quantify the accuracy and performance of the ICPINN method, we define the *Error* based on the MSE as follows:

$$Error = \frac{1}{N} \sum_{i=1}^N [V(\mathbf{x}^i) - \hat{V}(\mathbf{x}^i)]^2, \quad (3.1)$$

where $\{\mathbf{x}^i\}_{i=1}^N \subset \Omega$, N is the number of points, V represents the exact solution, and $\hat{V}$ denotes the predict solution.

If we use the *Error* to evaluate the performance of the ICPINN or other methods, the following points need to be noted:

- For the points $\{\mathbf{x}^i\}_{i=1}^N$, we choose equidistant sampling of the area Ω and reshape it into an array of dimensions (N, M) , where M is the dimension of Ω .
- To avoid excessive memory consumption during the iteration process, we check the *Error* every 40 iterations.
- It is crucial to note that the *Error* is not involved in the training process; it is included in the iterative optimization solely for monitoring purposes.
- When the PDE lacks a closed-form solution that can be substituted into Eq. (3.1), the approximate solution obtained by MC method will be used instead of the exact solution to compute the *Error*.

3.1. Example for Black–Scholes equation

We first consider solving the following standard BS equation [2]:

$$\mathcal{L}_{BS}V := \frac{\partial V}{\partial \tau} - \frac{\sigma^2 S^2}{2} \frac{\partial^2 V}{\partial S^2} - rS \frac{\partial V}{\partial S} + rV = 0, \quad (3.2)$$

where the parameters r and σ (both greater than zero) represent the risk-free interest rate and the volatility, $V(S, \tau)$ represents the value of the European option at the time remaining until maturity $\tau \in [0, T]$ with asset value S at that time, and the $\mathcal{L}_{BS}$ denotes differential operator. Moreover, T denotes the expiration time.

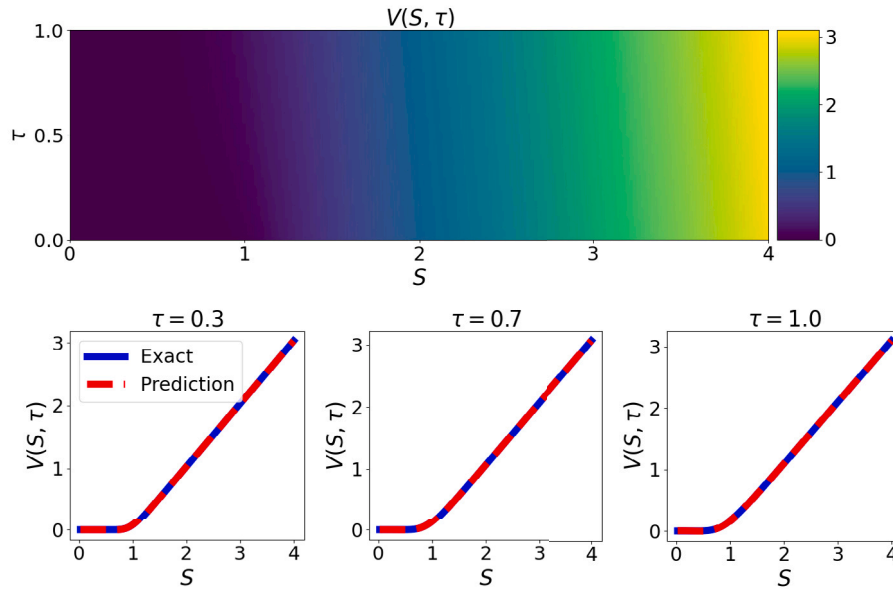


Fig. 3.1. The initial density plot depicts the solution derived from the ICPINN method. In the three plots situated below it, the red dashed line illustrates the predicted solution derived from the ICPINN method, whereas the blue solid line denotes the exact solution. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

At the expiration of the option $\tau = 0$, the price of the option is described by the payoff function $\max(S - K, 0)$, where the K denotes the strike price. When S is equal to zero, the option price is zero. As asset prices approach infinity, the option price increases at the same rate as the asset price. Thus the PDE (3.2) has the following BCs (3.3):

$$V(S, 0) = \max(S - K, 0), \quad (3.3a)$$

$$V(0, \tau) = 0, \quad (3.3b)$$

$$\lim_{S \rightarrow \infty} \frac{\partial V(S, \tau)}{\partial S} = 1. \quad (3.3c)$$

We need to note that when S is large enough, we truncate S and set the truncation value to $S_{\max}$. The parameter settings for the European call option are as follows:

$$K = 1, \quad r = 0.1, \quad \sigma = 0.3, \quad S_{\max} = 4K.$$

Then, we use a small feedforward neural network with hyperparameters from Table 4 to fit the Dirichlet BCs (3.3a) and (3.3b), and take the final training results as auxiliary function $A(S, \tau)$. For $B(S, \tau)$, we opt for the simplest form, $B(S, \tau) = S \times \tau$. The trial solution of the BS Eq. (3.2) is given by:

$$V_{\theta}(S, \tau) = A(S, \tau) + B(S, \tau) \times \text{NN}(S, \tau; \theta). \quad (3.4)$$

Next, if we set the parameters to $\lambda_r = \lambda_b = 1$, the PDE residual $\text{Loss}_r(\theta)$, the non-Dirichlet BCs residual $\text{Loss}_b(\theta)$ and the loss function $L_{\text{ICPINN}}(\theta)$ are constructed as follows:

$$\text{Loss}_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} [\mathcal{L}_{BS} V_{\theta}(S_r^i, \tau_r^i)]^2, \quad (3.5)$$

$$\text{Loss}_b(\theta) = \frac{1}{N_b} \sum_{i=1}^{N_b} \left[\frac{\partial V_{\theta}}{\partial S}(S_{\max}, \tau_b^i) - 1 \right]^2, \quad (3.6)$$

$$L_{\text{ICPINN}}(\theta) = \text{Loss}_b(\theta) + \text{Loss}_r(\theta), \quad (3.7)$$

where $N_r = 2000$, $N_b = 100$ and the hyperparameters of the neural network are shown in Table 3.

As shown in Fig. 3.1, the option price derived from the ICPINN method $V(S, \tau)$ is depicted by the initial density plot. To demonstrate more details of the predicted solution, we choose three different times $\tau = 0.3, 0.7, T$. From the point of view of the fitting effect of the predicted solution curve and the exact solution curve, we successfully captured the dynamics of the option price.

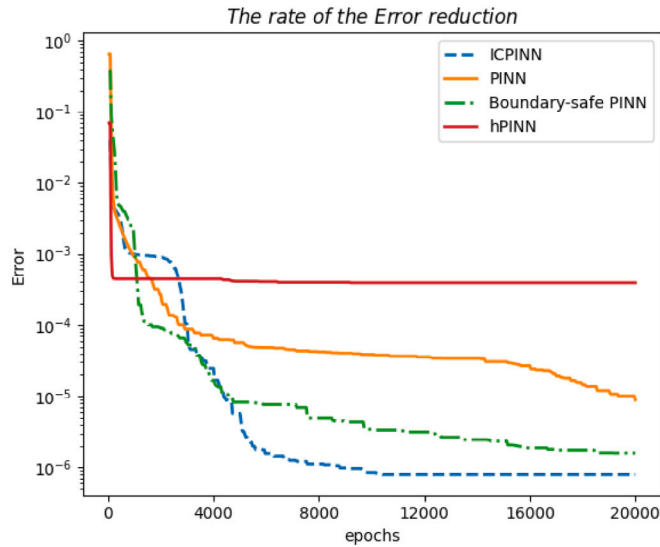


Fig. 3.2. The *Error*-decreasing iteration plot for various deep learning methods solving the BS model. In the plot, the blue line represents the results of the ICPINN method, the green line corresponds to the boundary-safe PINN method, the yellow line represents the standard PINN method, and the red line represents the hPINN method. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

In Fig. 3.1, the red dashed line represents the predicted solution obtained using the ICPINN method, while the blue line denotes the exact solution available in [40].

Although Fig. 3.1 demonstrates the accurate fitting capability of the ICPINN method to the BS model solution, it does not reflect the method's comparative performance with the same number of optimization steps. To quantify the performance and accuracy of the ICPINN algorithm, the *Error* defined in Eq. (3.1) is applied. We equidistantly sample 21 points ranging from 0 to $S_{\max}$ for S and 21 points ranging from 0 to T for τ to obtain (x^i, τ^i) values, forming a 21×21 grid. This grid is then reshaped into an array with dimensions $(21 \times 21, 2)$.

Fig. 3.2 shows the change of the *Error* value with the number of iterations, and the vertical axis represents the base-10 logarithm of the *Error* value. The blue line is obtained by the ICPINN method, the yellow line is given by the PINN method, the green line is given by the boundary-safe PINN method and the red line is given by the hPINN method. To objectively demonstrate the advantages of the ICPINN method, several methods adopt the hyperparameters in Table 3. Under these conditions, we observe that the result obtained by the hPINN method tends to become trapped in local minima, whereas the solution obtained through the ICPINN and PINN methods can converge to the exact solution. The final *Error* values are 7.9742×10^{-7} for the ICPINN method, 8.8611×10^{-6} for the PINN method and 3.1392×10^{-6} for boundary-safe PINN method, with the blue line consistently below the yellow line. From Fig. 3.2, it can be seen that the ICPINN method has a faster convergence speed and a higher accuracy. In addition, in Table 5, we directly compare the option prices at maturity obtained by the Finite Element Method [11], the Crank-Nicolson method [12], and the ICPINN method with the analytical solution at $S = 0.8, 1.2, 2.0$, and 2.6 . The results indicate that, for the one-dimensional BS model, the option prices obtained by the ICPINN method are comparable to those produced by other traditional methods.

3.2. Example for Heston option pricing PDE

In this part, we are considering solving the following Heston option pricing PDE, which is described by the following [4]:

$$\mathcal{L}_{Heston} U = \frac{1}{2} v S^2 \frac{\partial^2 U}{\partial S^2} + \rho \sigma v S \frac{\partial^2 U}{\partial S \partial v} + \frac{1}{2} v \sigma^2 \frac{\partial^2 U}{\partial v^2} + r S \frac{\partial U}{\partial S} + \kappa(\theta - v) \frac{\partial U}{\partial v} - rU - \frac{\partial U}{\partial \tau} = 0, \quad (3.8)$$

where $U(S, v, \tau)$ represents the value of the European option at the time remaining until maturity $\tau \in [0, T]$ with asset value S and variance v at that time, the $\mathcal{L}_{Heston}$ represents the differential operator, r is the risk-free rate, ρ is the correlation between the asset price and the variance, σ is the volatility of the variance, κ is the mean reversion of the variance, and θ is the long run mean of the variance. Moreover, T denotes the expiration time.

For a European call option satisfying a given PDE (3.8) with a strike price of K , the initial BC, or payoff condition is $U(S, v, 0) = \max(S - K, 0)$. The European call option satisfies $U(0, v, \tau) = 0$ at the boundary $S = 0$. When $v \rightarrow \infty$, the option price no longer changes. As asset prices approach infinity, the option price increases at the same rate as the asset price. Additionally, at $S \rightarrow \infty$ and

Table 5

Comparison among the solutions obtained by the ICPINN method, the Crank-Nicolson method, the Finite Element method, and the analytical solution at $\tau = 1$ in the one-dimensional Black-Scholes model. The absolute error represents the absolute difference between the analytical solution and the predicted solution.

S	0.8	1.2	2.0	2.6
Analytical	0.05758	0.32406	1.09567	1.49522
Finite Element	0.05759	0.32431	1.09576	1.49523
Absolute error	1×10^{-5}	2.5×10^{-4}	9×10^{-5}	1×10^{-5}
Crank-Nicolson	0.05739	0.32389	1.09565	1.49517
Absolute error	1.9×10^{-4}	1.7×10^{-4}	2×10^{-5}	5×10^{-5}
ICPINN	0.05755	0.32379	1.09560	1.49495
Absolute error	3×10^{-5}	2.7×10^{-4}	7×10^{-5}	2.7×10^{-4}

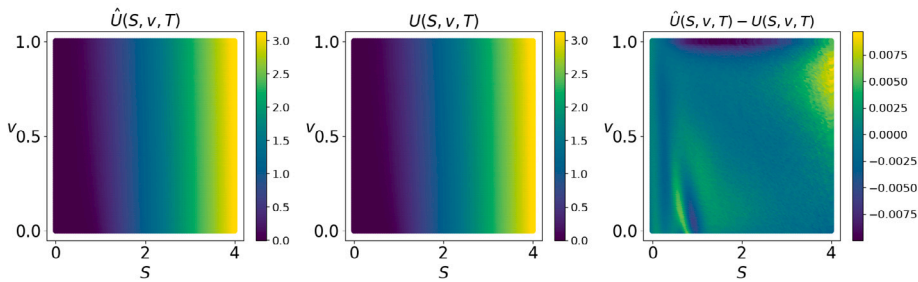


Fig. 3.3. When $\tau = T$, option prices $U(S, v, \tau)$, $\hat{U}(S, v, \tau)$ and their difference. The $\hat{U}(S, v, T)$ and $U(S, v, T)$ represent the predicted solution obtained by the ICPINN method and exact price, respectively. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

$v \rightarrow \infty$, we truncate to sufficiently large values, denoted as $S_{\max}$ and $v_{\max}$, respectively. In summary, the BCs for the Heston option pricing PDE are:

$$U(S, v, 0) = \max(S - K, 0), \quad (3.9a)$$

$$U(0, v, \tau) = 0, \quad (3.9b)$$

$$\lim_{S \rightarrow \infty} \frac{\partial U(S, v, \tau)}{\partial S} = 1, \quad (3.9c)$$

$$\lim_{v \rightarrow \infty} \frac{\partial U(S, v, \tau)}{\partial v} = 0. \quad (3.9d)$$

At $v = 0$, by considering the Fichera function, it is found that a degenerate BC is required only when the Feller condition $2\kappa\theta \geq \sigma^2$ is violated [41]. If $2\kappa\theta < \sigma^2$, the degenerate BC is given by:

$$\mathcal{B}_{deg}U = S \frac{\partial U}{\partial S} + \kappa\theta \frac{\partial U}{\partial v} - rU - \frac{\partial U}{\partial \tau} = 0, \quad (3.10)$$

where $\mathcal{B}_{deg}$ represents differential operator.

For parameters settings, we refer to Ref. [42]:

$$K = 1, \quad r = 0.1, \quad \kappa = 1, \quad \theta = 0.08, \quad \sigma = 0.39, \quad \rho = -0.93,$$

and the truncate values is $S_{\max} = 4 \times K$, $v_{\max} = 1$. Here we must point out that although our choice of parameters does not violate the Feller condition, omitting the degenerate boundary does not fundamentally affect the potential solution of the equation. As stated in Section 2, similar to other non-Dirichlet BCs, imposing the degenerate BC is helpful for numerical convergence, especially for the solution near that boundary. This is essential for the study of numerical algorithms. Moreover, most researchers include this BC when developing numerical methods for solving the Heston option pricing PDE [31,43,44].

As in the previous part, we use a small feedforward neural network with hyperparameters from Table 4 to fit Dirichlet BCs (3.9a) and (3.9b), and take the final training results as the auxiliary function $A(S, v, \tau)$. And we set $B(S, v, \tau) = S \times \tau$. The trial solution $U_{\theta}(S, v, \tau)$ is given by:

$$U_{\theta}(S, v, \tau) = A(S, v, \tau) + B(S, v, \tau) \times \text{NN}(S, v, \tau; \theta). \quad (3.11)$$

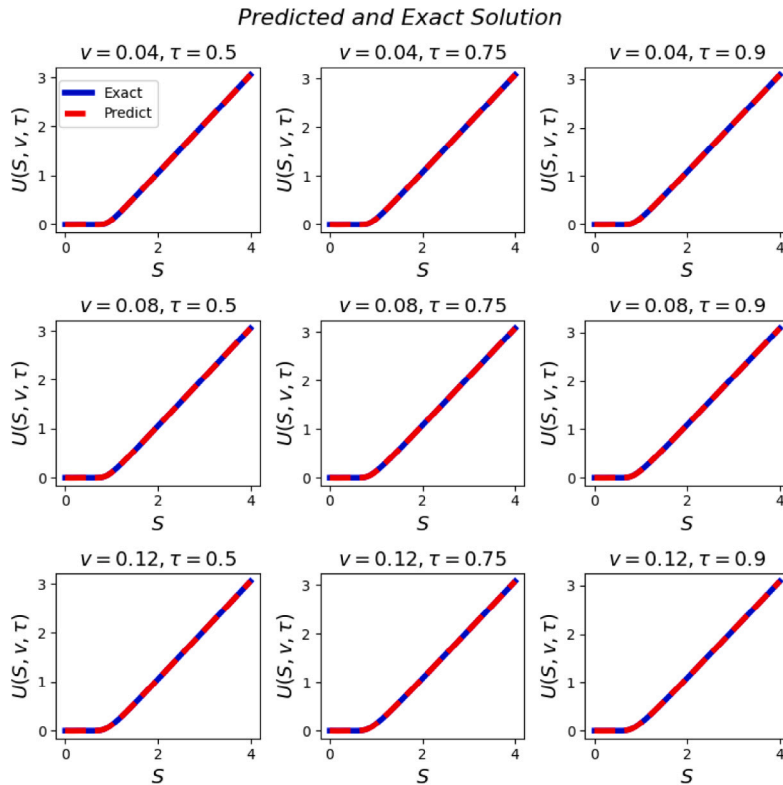


Fig. 3.4. When $\tau = 0.5, 0.75, 0.9$, the plot of option prices with closing to reality volatility v . The red dashed line in the graph represents the predicted solution derived from the ICPINN method, while the blue solid line represents the exact solution. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Then, based on setting $\lambda_r = \lambda_{S_{\max}} = \lambda_{v_{\max}} = \lambda_{deg} = 1$, the loss function can be constructed as follows:

$$Loss_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} (\mathcal{L}_{Heston} U_\theta(S_r^i, v_r^i, \tau_r^i))^2, \quad (3.12)$$

$$L_{s_{\max}}(\theta) = \frac{1}{N_{s_{\max}}} \sum_{i=1}^{N_{s_{\max}}} \left(\frac{\partial U_\theta}{\partial S}(S_{\max}^i, v_{s_{\max}}^i, \tau_{s_{\max}}^i) - 1 \right)^2, \quad (3.13)$$

$$L_{v_{\max}}(\theta) = \frac{1}{N_{v_{\max}}} \sum_{i=1}^{N_{v_{\max}}} \left(\frac{\partial U_\theta}{\partial v}(S_{v_{\max}}^i, v_{\max}^i, \tau_{v_{\max}}^i) \right)^2, \quad (3.14)$$

$$L_{deg}(\theta) = \frac{1}{N_{deg}} \sum_{i=1}^{N_{deg}} (\mathcal{B}_{deg} U_\theta(S_{deg}^i, 0, \tau_{deg}^i))^2, \quad (3.15)$$

$$L_{ICPINN}(\theta) = Loss_r(\theta) + L_{s_{\max}}(\theta) + L_{v_{\max}}(\theta) + L_{deg}(\theta). \quad (3.16)$$

In our configuration, we have $N_r = 10000$ interior points and $N_{s_{\max}} = N_{v_{\max}} = N_{v_{\min}} = 500$ boundary points.

It is worth noting that although our parameter selection conforms to the Feller condition, in the ICPINN method, we do not explicitly check for its violation to ensure methodological completeness. In other words, we do not omit the residual term (3.15) regardless, even though having fewer terms in the loss function (3.16) might offer advantages in minimizing loss.

We first present the predicted solution $\hat{U}(S, v, T)$ obtained by the ICPINN method compared to the exact solution $U(S, v, T)$ from [4] at $\tau = T$, along with the result of their subtraction $\hat{U}(S, v, T) - U(S, v, T)$, as shown in Fig. 3.3. The results demonstrate that the ICPINN method effectively captures the solution to the Heston option pricing PDE. Moreover, to demonstrate the effectiveness of our method more comprehensively and the details of the predicted solution, we consider three different times $\tau = 0.5, 0.75, 0.9$. Since real variance is typically not very high, we chose to test cases with $v = 0.04, 0.08, 0.12$ as illustrated in Fig. 3.4. Additionally, a large v value is necessary when testing the effectiveness of our method, we chose to test cases with $v = 0.2, 0.4, 0.8$ as depicted in Fig. 3.5. In Figs. 3.4 and 3.5, the blue solid line represents the exact solution and the red dashed line represents the predicted

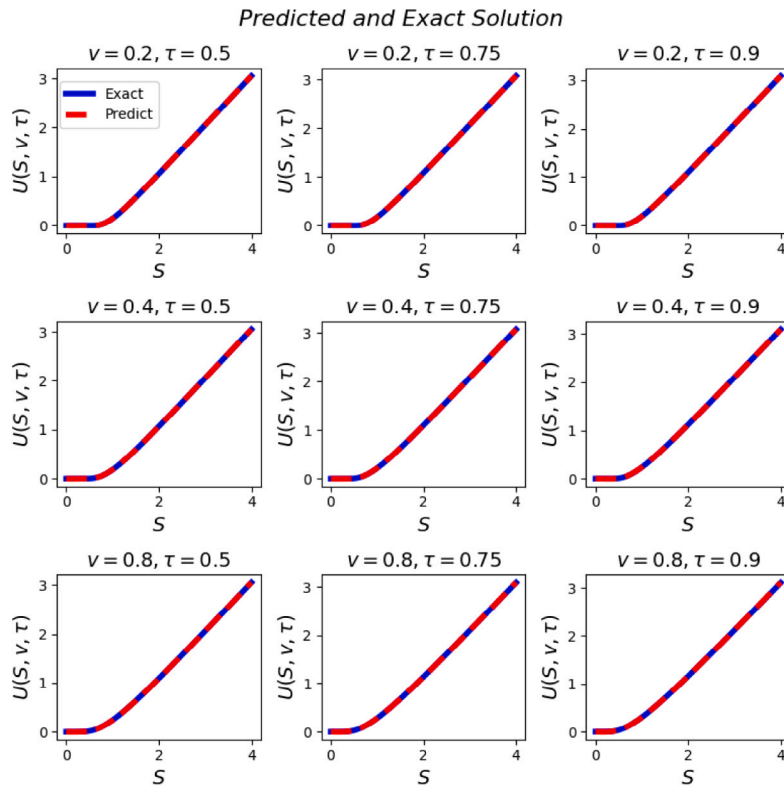


Fig. 3.5. When $\tau = 0.5, 0.75, 0.9$, the plot of option prices with large volatility v . The red dashed line in the graph represents the predicted solution derived from the ICPINN method, while the blue solid line represents the exact solution. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

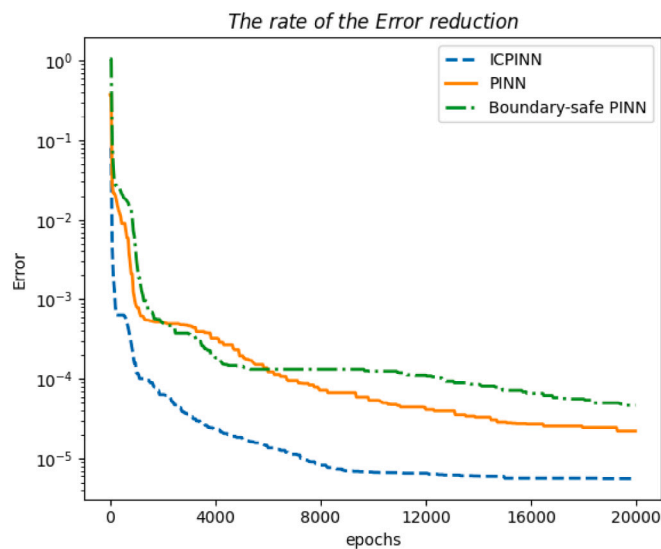


Fig. 3.6. The *Error*-decreasing iteration plot for various deep learning methods solving the Heston option pricing PDE. In the plot, the blue line represents the results of the ICPINN method, the green line corresponds to the boundary-safe PINN method, and the yellow line represents the standard PINN method. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 6

Comparison among the solutions obtained by the ICPINN method, the Crank-Nicolson method, the Finite Element method, and the analytical solution at $\tau = 1$ in the Heston option pricing PDE. The absolute error represents the absolute difference between the analytical solution and the predicted solution.

S	0.8	1.2	2.0	2.6
$v = 0.05$				
Analytical	0.02542	0.32124	1.09803	1.69604
Finite Element	0.02549	0.32130	1.09888	1.69544
Absolute error	7×10^{-5}	6×10^{-5}	8.5×10^{-4}	6×10^{-4}
Crank-Nicolson	0.02541	0.32155	1.09875	1.69606
Absolute error	1×10^{-5}	3.1×10^{-4}	7.2×10^{-4}	2×10^{-5}
ICPINN	0.02492	0.32213	1.09737	1.69517
Absolute error	5×10^{-4}	8.9×10^{-4}	6.6×10^{-4}	8.7×10^{-4}
$v = 0.5$				
Analytical	0.13507	0.41988	1.14045	1.72029
Finite Element	0.13513	0.41991	1.14021	1.71995
Absolute error	6×10^{-5}	3×10^{-5}	2.4×10^{-4}	3.4×10^{-4}
Crank-Nicolson	0.13506	0.41999	1.14047	1.72016
Absolute error	1×10^{-5}	1.1×10^{-4}	2×10^{-5}	1.3×10^{-4}
ICPINN	0.13615	0.41946	1.13919	1.71874
Absolute error	5×10^{-4}	8.9×10^{-4}	6.6×10^{-4}	8.7×10^{-4}

solution obtained by the ICPINN method. The results indicate that the ICPINN method can perfectly predict the value of short-term, medium-term, and long-term options.

Then, to demonstrate the superior performance of the ICPINN method, the *Error* is applied. For (x^i, v^i, τ^i) values, we uniformly sample 21 points ranging from 0 to S_{max} for x , 0 to v_{max} for v , and from 0 to T for τ , forming a $21 \times 21 \times 21$ grid. We then reshape this grid into an array with dimensions $(21^3, 3)$.

During the training process, we obtain the *Error*-decreasing curve as shown in Fig. 3.6, which demonstrates the change of the *Error* value with the number of iterations. The vertical axis represents the base-10 logarithm of the *Error* value and the final *Error* value of the ICPINN method is 5.622×10^{-6} , the PINN method is 2.2219×10^{-5} and the boundar-safe PINN method is 4.6491×10^{-5} . The ICPINN method obtains the blue line, the yellow line is given by the PINN method and the red line is given by the PINN method. To objectively demonstrate the advantages of the ICPINN method, several methods adopt the hyperparameters in Table 3. In Fig. 3.6, the blue line below the yellow line always demonstrates that the ICPINN method converges faster and has higher accuracy at the same number of iteration steps. Moreover, for the Heston option pricing PDE, satisfying the degenerate BC through trial solutions proves challenging, making hPINN approaches nearly ineffective in this scenario. The same as the previous part, in Table 6, we directly compare the solutions of the Heston option pricing PDE at maturity obtained by the Finite Element Method [11], the Crank-Nicolson method [12], and the ICPINN method with the analytical solution for $v = 0.05$ and 0.5 , $S = 0.8, 1.2, 2.0$, and 2.6 . The results indicate that, for the one-dimensional BS model, the option prices obtained by the ICPINN method are comparable to those produced by other traditional methods.

3.3. Three-asset Black-Scholes equation

To further investigate the effectiveness of the ICPINN algorithm, we examine in this section a representative and challenging case: the BS option pricing PDE for three underlying assets in a four-dimensional spatio-temporal setting.

The three-asset BS model is formulated as follows:

$$\mathcal{L}_{BS} V = \frac{\partial V}{\partial \tau} - \frac{1}{2} \sum_{i=1}^3 \sum_{j=1}^3 \rho_{ij} \sigma_i \sigma_j S_i S_j \frac{\partial^2 V}{\partial S_i \partial S_j} - r \sum_{i=1}^3 S_i \frac{\partial V}{\partial S_i} + rV = 0, \quad (3.17)$$

where r denotes the risk-free interest rate, and σ_1, σ_2 and σ_3 represent the volatilities of the three assets. All parameters r and σ_i ($i = 1, 2, 3$) are strictly positive real numbers. The function $V = V(S_1, S_2, S_3, \tau)$ represents the value of a European option on three underlying assets. Moreover, the $\mathcal{L}_{BS}$ denotes the differential operator, T is the expiration time, and ρ_{ij} ($i, j = 1, 2, 3$) are the correlation coefficients between the assets, satisfying $\rho_{ij} = \rho_{ji}$.

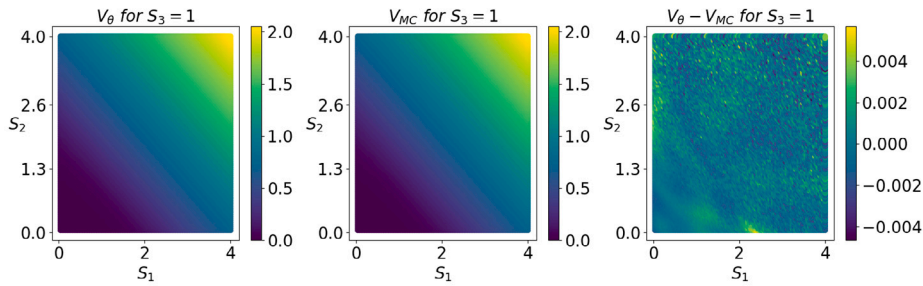


Fig. 3.7. Option prices V_θ , V_{MC} and their difference with respect to assets S_1 and S_2 when $\tau = T$ and $S_3 = 1$. Here, V_θ represents the predicted solution obtained by the ICPINN algorithm, while V_{MC} denotes the solution computed using the Monte Carlo method. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

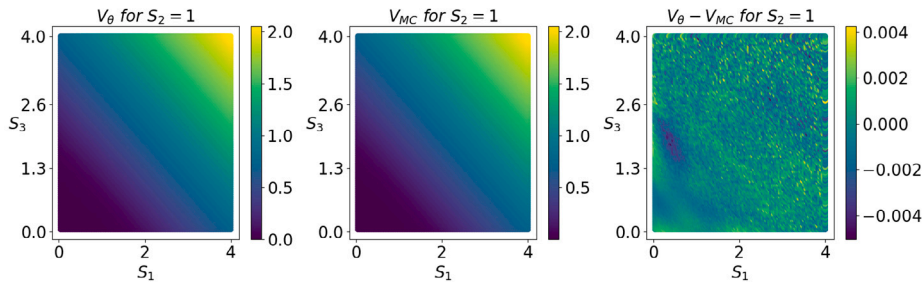


Fig. 3.8. Option prices V_θ , V_{MC} and their difference with respect to assets S_1 and S_3 when $\tau = T$ and $S_2 = 1$. Here, V_θ represents the predicted solution obtained by the ICPINN algorithm, while V_{MC} denotes the solution computed using the Monte Carlo method. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

At $\tau = 0$, various option types exist within the three-asset framework. This study focuses on the arithmetic average basket option, whose payoff function is defined as:

$$\text{Payoff} = \max \left(\frac{S_1 + S_2 + S_3}{3} - K, 0 \right), \quad (3.18)$$

where K denotes the strike price.

Unlike the single-asset case, in the three-asset BS model, when $S_i = 0$, the option price depends on the remaining assets S_j , where $j \neq i$. In other words, at this boundary, a Dirichlet BC can be directly applied, where the option price is equal to the solution of the corresponding two-asset BS model. However, the solution to the two-asset BS model remains unknown. Indeed, many scholars employ different boundary conditions to study the solutions of these option pricing PDEs [45,46], and our application of non-Dirichlet boundary conditions does not fundamentally alter the potential solutions. However, as stated in Section 1, we emphasize that for option pricing models seeking higher accuracy and faster convergence, solving the equations with non-Dirichlet boundary conditions proves more effective [1]. Therefore, we substitute $S_i = 0$ into the original PDE to derive the associated degenerate boundary condition, which can then be used to aid in solving the full three-dimensional problem.

Specifically, we impose the degenerate boundary condition at $S_k = 0$ as follows:

$$\mathcal{B}_{\text{deg}}^{S_k} V = \frac{\partial V}{\partial \tau} - \frac{1}{2} \sum_{i=1}^3 \sum_{j=1, j \neq k}^3 \rho_{ij} \sigma_i \sigma_j S_i S_j \frac{\partial^2 V}{\partial S_i \partial S_j} - r \sum_{i=1, i \neq k}^3 S_i \frac{\partial V}{\partial S_i} + rV. \quad (3.19)$$

Here, $\mathcal{B}_{\text{deg}}^{S_k}$ ($k = 1, 2, 3$) denotes the differential operator corresponding to the degenerate boundary condition at $S_k = 0$.

As the asset prices S_k tend to infinity, the price of the three-asset option becomes increasingly dominated by S_k . According to the discounting principle, the option price at this boundary satisfies a Dirichlet boundary condition, given by:

$$V(S_1, S_2, S_3, \tau) = \frac{S_1 + S_2 + S_3}{3} - K \cdot e^{-r\tau}, \quad \text{as } S_k \rightarrow \infty. \quad (3.20)$$

For numerical implementation, the domain is truncated at S_k at $S_{\max}$, such that $(S_1, S_2, S_3) \in [0, S_{\max}]^3$. Furthermore, the Dirichlet BCs (3.21) are imposed at the truncated boundaries, $S_k = S_{\max}$, namely:

$$V(S_1, S_2, S_3, \tau) = \frac{S_1 + S_2 + S_3}{3} - K \cdot e^{-r\tau}, \quad \text{as } S_k = S_{\max}. \quad (3.21)$$

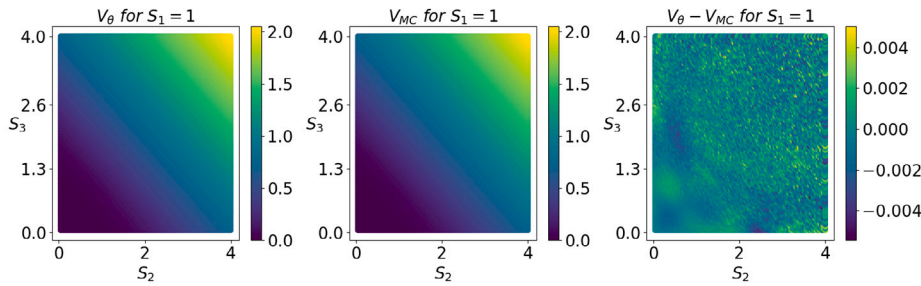


Fig. 3.9. Option prices V_θ , V_{MC} and their difference with respect to assets S_2 and S_3 when $\tau = T$ and $S_1 = 1$. Here, V_θ represents the predicted solution obtained by the ICINN algorithm, while V_{MC} denotes the solution computed using the Monte Carlo method. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

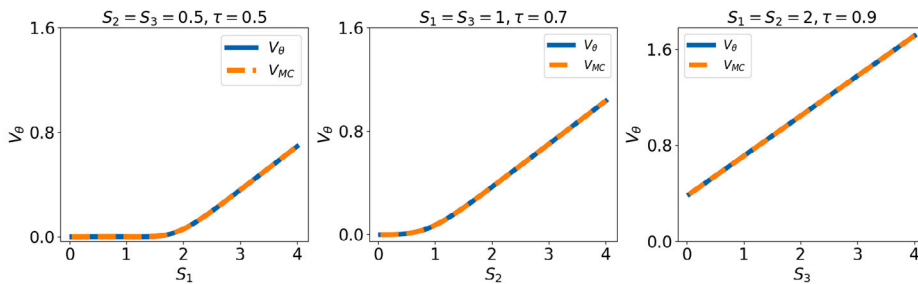


Fig. 3.10. Three plots of option prices varying with asset values. In the first plot, the option price varies with S_1 , where $S_2 = S_3 = 0.5$, and $\tau = 0.5$. In the second plot, the option price varies with S_2 , where $S_1 = S_3 = 1$, and $\tau = 0.7$. In the third plot, the option price varies with S_3 , where $S_1 = S_2 = 2$, and $\tau = 0.9$. The blue line represents the results obtained using the ICINN method, while the yellow line indicates the reference solution obtained via Monte Carlo simulation. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Once the expression of the BCs is determined, we proceed to construct the trial solution $V_\theta(S_1, S_2, S_3, \tau)$. As in the previous two sections, in this part, we also use a small neural network to approximate the Dirichlet BCs (3.18) and (3.21). The parameters of the neural network are shown in Table 4, and we denote the final training result as $A(S_1, S_2, S_3, \tau)$. It is worth noting that, in this section, our setup for the function $B(S_1, S_2, S_3, \tau)$ differs slightly from the previous sections. Firstly, the Dirichlet BCs are now applied on the upper boundary, while the non-Dirichlet BCs are applied on the lower boundary. Secondly, to mitigate the numerical explosion caused by the four-dimensional space, we define $B(S_1, S_2, S_3, \tau)$ in the following form:

$$B(S_1, S_2, S_3, \tau) = \frac{\tau(S_{\max} - S_1)(S_{\max} - S_2)(S_{\max} - S_3)}{T S_{\max}^3}. \quad (3.22)$$

Unless otherwise specified, our parameters are set as follows:

$$\begin{aligned} K = 1, T = 1, r = 0.05, S_{\max} = 4K, \sigma_1 = 0.2, \sigma_2 = 0.3, \sigma_3 = 0.25, \\ \rho_{12} = \rho_{21} = 0.1, \rho_{13} = \rho_{31} = 0.2, \rho_{23} = \rho_{32} = 0.15. \end{aligned} \quad (3.23)$$

Therefore, based on PDE (3.17) with non-Dirichlet BCs (3.19), we construct the residual and loss function as follows:

$$Loss_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} (\mathcal{L}_{BS} V_\theta(S_1^i, S_2^i, S_3^i, \tau^i))^2, \quad (3.24)$$

$$Loss_{S_1} = \frac{1}{N_{S_1}} \sum_{i=1}^{N_{S_1}} (\mathcal{B}_{deg}^{S_1} V_\theta(0, S_2^i, S_3^i, \tau^i))^2, \quad (3.25)$$

$$Loss_{S_2} = \frac{1}{N_{S_2}} \sum_{i=1}^{N_{S_2}} (\mathcal{B}_{deg}^{S_2} V_\theta(S_1^i, 0, S_3^i, \tau^i))^2, \quad (3.26)$$

$$Loss_{S_3} = \frac{1}{N_{S_3}} \sum_{i=1}^{N_{S_3}} (\mathcal{B}_{deg}^{S_3} V_\theta(S_1^i, S_2^i, 0, \tau^i))^2, \quad (3.27)$$

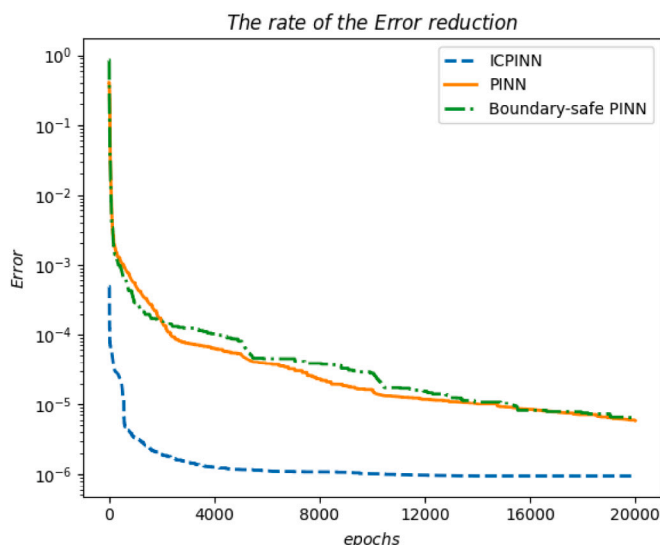


Fig. 3.11. The *Error*-decreasing iteration plot for various deep learning methods solving the three-asset BS model. In the plot, the blue line represents the results of the ICPINN method, the green line corresponds to the boundary-safe PINN method, and the yellow line represents the standard PINN method. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 7

The computation times of the MC simulation and the ICPINN methods with 4 or 8 hidden-layers under the three-asset BS model.

	r=0.05 T=1.0	r=0.05 T=1.5	r=0.15 T=1.0	r=0.15 T=1.5	r=0.25 T=1.0	r=0.25 T=1.5
MC	2423.4s	2396.6s	2364.7s	2456.2s	2432.2s	2388.5s
8-layers	1324.2s	1289.3s	1265.6s	1355.9s	1320.1s	1311.8s
<i>Error</i> ($\times 10^{-7}$)	9.2721	8.3627	12.5646	8.7512	8.6754	10.8651
4-layers	256.8s	234.1s	243.2s	286.4s	228.9s	258.2s
<i>Error</i> ($\times 10^{-6}$)	5.5627	6.7128	4.3762	7.9832	5.8537	2.1875

where $\{S_1^i, S_2^i, S_3^i, \tau\} \in (0, S_{\max})^3 \times (0, T)$. In our configuration, we have $N_r = 15000$ interior points and $N_{S_i} = 1000$ ($i = 1, 2, 3$) boundary points.

To evaluate the performance of the results obtained by the ICPINN method, we compare them with extremely reliable approximations of the desired solutions computed by Monte Carlo method. Figs. 3.7–3.9 illustrate the relationship between the predicted option price and the remaining two assets when one of S_1 , S_2 , or S_3 is fixed, given $\tau = T$. Here, V_θ represents the predicted solution obtained by the ICPINN algorithm, while V_{MC} denotes the reference solution obtained using the Monte Carlo method. Although the Monte Carlo method has an error of $O(1/\sqrt{N})$ compared to the exact solution, the difference $V_\theta - V_{MC}$ remains around 10^{-3} , demonstrating the effectiveness of the ICPINN method in solving four-dimensional PDEs.

Furthermore, to validate the effectiveness of our solution across multiple time scales, Fig. 3.10 illustrates the variation of V_θ and V_{MC} with respect to S_i ($i = 1, 2, 3$) at $\tau = 0.5, 0.7$, and 0.9 , respectively. The results indicate that the ICPINN method consistently aligns with the Monte Carlo method across all three time instances. This further demonstrates that the ICPINN approach can effectively capture the solution of the three-asset arithmetic average basket BS option PDE throughout the entire spatiotemporal domain.

To demonstrate the superior performance of the ICPINN method, the *Error* is applied. It should be noted that in Eq. (3.1), the exact solution is replaced with a Monte Carlo approximation based on 100,000 simulated paths. For the values of $(S_1^i, S_2^i, S_3^i, \tau^i)$, we uniformly sampling $21 \times 21 \times 21$ points within $[0, S_{\max}]^3$ and 21 points within $[0, T]$. This results in a four-dimensional spatiotemporal grid of size $21 \times 21 \times 21 \times 21$, which is then reshaped into an array of shape $(21^4, 4)$.

During the training process, we obtain the *Error*-decreasing curve as shown in Fig. 3.11, which demonstrates the change of the *Error* value with the number of iterations. The vertical axis represents the base-10 logarithm of the *Error* value, and the final *Error* values are 9.2721×10^{-7} for the ICPINN method, 3.3185×10^{-6} for the PINN method, and 2.4574×10^{-6} for the boundary-safe PINN method. In Fig. 3.11 the ICPINN results (blue curve) demonstrate superior convergence compared to the boundary-safe PINN (green curve) and standard PINN (yellow curve) methods. The neural network configurations for all three deep learning methods

are provided in Table 3. Since the trial solution is preprocessed to satisfy the Dirichlet BCs, the initial *Error* of ICPINN is lower than that of other deep learning methods. Moreover, we observe that after 10,000 training epochs, the *Error* decreases only marginally, suggesting that there is still room for improvement in the tuning of the neural networks hyperparameters.

Table 7 presents a comparative analysis of the computational time and accuracy of the MC method versus the ICPINN approach. Within the ICPINN framework, neural networks comprising 4 and 8 hidden layers are evaluated, with both the training time and associated error metrics reported for each configuration. The results demonstrate that, in the four-dimensional setting, maintaining the same hyperparameters as in the two-dimensional case while employing an 8-layer network yields an acceptable error. However, this configuration incurs a significantly higher training cost. Reducing the network depth to 4 layers results in a substantial decrease in computational time, accompanied by only a marginal increase in error, which remains within an acceptable threshold. This highlights a favorable trade-off between accuracy and efficiency in four-dimensional scenarios.

4. Conclusions

In this paper, we introduce the ICPINN method, a novel deep-learning approach that incorporates distinct constraints tailored for various BCs. We provide a comprehensive overview of the ICPINN method, detailing its implementation process. Furthermore, we demonstrate the effectiveness of the ICPINN method by applying it to solve the BS model, the Heston option pricing PDE and three-asset BS model. Our findings indicate that the ICPINN method achieves accurate convergence to solutions for option pricing PDEs and exhibits significantly improved convergence speed.

While our primary focus in this article centers on European options, it's important to note that American options, as prevalent free boundary problems in the market, remain crucial. Our future direction will concentrate on extending the ICPINN method to tackle such free boundary problems, which promises to be an intriguing endeavor.

Acknowledgments

This research is supported with fund provided by the National Natural Science Foundation of China (No. 12371402). The authors extend sincere appreciation to the reviewers for their expert guidance. Their recommendations were pivotal in refining the manuscript's rigor.

CRediT authorship contribution statement

Jianguo Tan: Resources, Project administration, Methodology. **Xingyu Zhang:** Writing – review & editing, Writing – original draft, Validation, Software, Resources, Methodology.

References

- [1] J.P. Villarino, Á. Leito, J.A.G. Rodríguez, Boundary-safe PINNs extension: Application to non-linear parabolic PDEs in counterparty credit risk, *J. Comput. Appl. Math.* 425 (2003) 115041.
- [2] F. Black, M. Scholes, The pricing of options and corporate liabilities, *J. Political Econ.* 81 (1973) 637–654.
- [3] L. Wu, Y.K. Kwok, A front-fixing finite difference method for the valuation of American options, *J. Financ. Eng.* 6 (1997) 83–97.
- [4] S.L. Heston, A closed-form solution for options with stochastic volatility with applications to bond and currency options, *Rev. Financ. Stud.* 6 (1993) 327–343.
- [5] I. Florescu, M.C. Mariani, I. Sengupta, Option pricing with transaction costs and stochastic volatility, *Electron. J. Differential Equations* 2014 (2014) 1–19.
- [6] S. Ikonen, J. Toivanen, Efficient numerical methods for pricing American options under stochastic volatility, *Numer. Methods Partial. Differ. Equ.: Int. J.* 24 (2008) 104–126.
- [7] W. Margrabe, The value of an option to exchange one asset for another, *J. Finance.* 33 (1978) 177–186.
- [8] C. Alexander, A. Venkatraman, Analytic approximations for multi-asset option pricing, *Math. Finance.* 22 (2012) 667–689.
- [9] R.C. Merton, Option pricing when underlying stock returns are discontinuous, *J. Finance.* 3 (1976) 125–144.
- [10] S.G. Kou, A jump-diffusion model for option pricing, *Manag. Sci.* 48 (2002) 1086–1101.
- [11] Y. Achdou, O. Pironneau, *Computational Methods for Option Pricing*, Society for Industrial and Applied Mathematics, 2005.
- [12] P. Wilmott, J. Dewynne, S. Howison, *Option Pricing: Mathematical Models and Computation*, Oxford Financial Press, 1993.
- [13] R. Cont, E. Voltchkova, A finite difference scheme for option pricing in jump diffusion and exponential Lévy models, *SIAM J. Numer. Anal.* 43 (2005) 1596–1626.
- [14] L. Jiang, C. Li, *Mathematical Modeling and Methods of Option Pricing*, World Scientific, 2005.
- [15] P. Carr, D. Madan, Option valuation using the fast Fourier transform, *J. Comput. Finance.* 2 (1999) 61–73.
- [16] P. Glasserman, N. Merener, Numerical solution of jump-diffusion LIBOR market models, *Finance Stoch.* 7 (2003) 1–27.
- [17] A. Itkin, P. Carr, Using pseudo-parabolic and fractional equations for option pricing in jump diffusion models, *Comput. Econ.* 40 (2012) 63–104.
- [18] L. Andersen, J. Andreasen, Jump-diffusion processes: Volatility smile fitting and numerical methods for option pricing, *Rev. Deriv. Res.* 4 (2000) 231–262.
- [19] W. Jia, X. Feng, M. Hao, Deep neural network method to predict the dynamical system response under random excitation of combined Gaussian and poisson white noises, *Chaos Solit. Fractals* 185 (2024) 115134.
- [20] R.T.Q. Chen, Y. Rubanova, J. Bettencourt, Neural ordinary differential equations, *Adv. Neural Inf. Process. Syst.* 31 (2018) 6572–6583.
- [21] H.H.N. Nguyen, T. Nguyen, H. Vo, Improving neural ordinary differential equations with Nesterov's accelerated gradient method, *Adv. Neural Inf. Process. Syst.* 35 (2022) 7712–7726.
- [22] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics informed deep learning (part I): Data-driven solutions of nonlinear partial differential equations, 2017, arXiv preprint arXiv:1711.10561.
- [23] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.

- [24] A.D. Jagtap, K. Kawaguchi, G.E. Karniadakis, Locally adaptive activation functions with slope recovery for deep and physics informed neural networks, *Proc. R. Soc. A* 476 (2020) 20200334.
- [25] J. Han, A. Jentzen, Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations, *Commun. Math. Stat.* 5 (2017) 349–380.
- [26] G. Pang, L. Lu, G.E. Karniadakis, fPINNs: Fractional physics-informed neural networks, *SIAM J. Sci. Comput.* 41 (2019) A2603–A2626.
- [27] Z. Hu, K. Kawaguchi, Z. Zhang, Tackling the curse of dimensionality in fractional and tempered fractional PDEs with physics-informed neural networks, *Comput. Methods Appl. Mech. Eng.* 432 (2024) 117448.
- [28] L. Yuan, Y.Q. Ni, X.Y. Deng, A-PINN: Auxiliary physics informed neural networks for forward and inverse problems of nonlinear integro-differential equations, *J. Comput. Phys.* 462 (2022) 111260.
- [29] Y. Bai, T. Chaolu, S. Bilige, The application of improved physics-informed neural network (IPINN) method in finance, *Nonlinear Dynam.* 107 (2022) 3655–3667.
- [30] K. Glau, L. Wunderlich, The deep parametric PDE method and applications to option pricing, *Appl. Math. Comput.* 432 (2022) 127355.
- [31] D. Hainaut, A. Casas, Option pricing in the Heston model with physics inspired neural networks, *Ann. Fin.* 20 (2024) 353–376.
- [32] L. Lu, R. Pestourie, W. Yao, Physics-informed neural networks with hard constraints for inverse design, *SIAM J. Sci. Comput.* 43 (2021) B1105–B1132.
- [33] I.E. Lagaris, A. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, *IEEE Trans. Neural Netw.* 9 (1998) 987–1000.
- [34] X. Wang, J. Li, J. Li, A deep learning based numerical PDE method for option pricing, *Comput. Econ.* 62 (2023) 149–164.
- [35] D.S. Santos, T.A.E. Ferreira, Neural network learning of Black–Scholes equation for option pricing, 2024, *arXiv preprint arXiv:2405.05780*.
- [36] M. Hou, H. Fu, Z. Hu, Numerical solving of generalized Black–Scholes differential equation using deep learning based on blocked residual connection, *Digit. Signal Process.* 126 (2022) 103498.
- [37] S. Eskiizmirli, K. Günel, R. Polat, On the solution of the Black–Scholes equation using feed-forward neural networks, *Comput. Econ.* 58 (2021) 915–941.
- [38] N. Umeorah, J.C. Mba, Approximation of single-barrier options partial differential equations using feed-forward neural network, *Appl. Stoch. Models Bus. Ind.* 38 (2022) 1079–1098.
- [39] I.M. Sobol, On the distribution of points in a cube and the approximate evaluation of integrals, *Zhurnal Vychislitel'noi Mat. i Mat. Fiz.* 7 (1967) 784–802.
- [40] C.C.W. Leentvaar, Numerical Solution of the Black–Scholes Equation with a Small Number of Grid Points, Delft University of Technology, 2003, Faculty of Electrical Engineering, Mathematics and Computer Science, Section Applied Mathematics, Numerical Analysis Group.
- [41] C. O'Sullivan, S. O'Sullivan, Pricing European and American options in the Heston model with accelerated explicit finite differencing methods, *Int. J. Theor. Appl. Finance* 16 (2013) 1350015.
- [42] A. Ribeiro, R. Poulsen, Approximation behaves calibration, *Quant. Financ. Lett.* 1 (2013) 36–40.
- [43] Q. Zhang, H. Song, Y. Hao, Semi-implicit FEM for the valuation of American options under the Heston model, *Comput. Appl. Math.* 41 (2022) 73.
- [44] X. Lu, S.P. Zhang, D. Yan, Nonlinear PDE model for European options with transaction costs under Heston stochastic volatility, *Commun. Nonlinear Sci. Numer. Simul.* 103 (2021) 105986.
- [45] J. Kim, T. Kim, J. Jo, A practical finite difference method for the three-dimensional Black–Scholes equation, *Eur. J. Oper. Res.* 252 (2016) 183–190.
- [46] N.M. Ahmed, F. Soleymani, R.K. Saeed, Numerical investigation of high-dimensional option pricing PDEs by utilizing a hybrid radial basis function – finite difference procedure, *Eng. Anal. Bound. Elem.* 163 (2024) 211–222.